차 례

Blacklist 오프라인 설치 가이드 **1**
1. 설치 전 요구사항 1
2. 패키지 확인과 압축 해제 1
3. 원커맨드 설치 1
4. 단계별 설치 2
5. 설치 결과 확인 3
6. WARP 프록시 3
7. 업그레이드 3
8. 데이터 포함 패키지 4

Blacklist 오프라인 설치 가이드

1. 설치 전 요구사항

- Linux x86_64 호스트
- root 또는 sudo 권한
- Docker Engine 과 Docker Compose v2 (패키지에 포함되지 않으므로 미리 설치)
- HTTPS 443 포트 사용 가능
- Docker 데이터 디렉터리에 충분한 여유 공간
- sha256sum, openssl, python3, curl, tar, unzip, gpg, gpgv, ip, ss (ip 와 ss 는 iproute2 패키지)
- 조직의 인증된 별도 채널로 받은 릴리스 공개키와 40 자리 fingerprint
- 실제 접속 FQDN 또는 IP 를 SAN 에 포함한 프론트엔드 TLS 서버 인증서와 개인키 (PEM)

현재 패키지는 Docker 와 Compose 가 이미 설치된 호스트를 기본 전제로 합니다. prereqs/에 Docker tarball 과 Compose 바이너리가 모두 들어 있는 패키지만 베어 호스트 설치를 지원합니다.

공개키, 인증서, 개인키는 압축을 푼 패키지 디렉터리 밖에 둡니다. 설치기는 MANIFEST.sha256 에 없는 파일이 패키지 디렉터리 안에 있으면 설치를 중단합니다.

2. 패키지 확인과 압축 해제

ZIP 패키지는 함께 배포된 SHA-256 파일로 전송 무결성을 확인한 뒤 표준 unzip 으로 풉니다. ZIP 의 진위는 설치기가 내부 MANIFEST.sha256.asc 서명으로 검증합니다.

```
sha256sum -c blacklist-<버전>-release.zip.sha256
unzip blacklist-<버전>-release.zip
cd blacklist-<버전>
```

tarball 을 받은 경우에는 4 장의 방법으로 공개키를 먼저 등록하고, 압축을 풀기 전에 최종 tarball 서명을 검증합니다.

```
gpgv --keyring /etc/blacklist/release-pubkey.gpg blacklist-<버전>.tar.gz.asc blacklist-<버전>.tar.gz
sha256sum -c blacklist-<버전>.tar.gz.sha256
tar -xzf blacklist-<버전>.tar.gz
cd blacklist-<버전>
```

서명 또는 체크섬 검증이 실패하면 압축을 풀거나 설치기를 실행하지 않습니다.

3. 원커맨드 설치

설치 전에 네트워크만 읽기 전용으로 점검할 수 있습니다. 인터페이스와 할당 주소, 실제 적용할 바인딩, 443 포트 충돌, 호스트 및 Docker 서브넷 중복, 내부 서브넷·프론트엔드 IP·신뢰 프록시 설정의 일관성을 확인한 뒤 종료합니다. 파일이나 컨테이너는 변경하지 않습니다.

```
sudo bash install.sh --pre-scan
```

주소 선택 절차까지 미리 보려면 다음 명령을 사용합니다. 사전 점검, 인터페이스 선택, 해당 인터페이스에 이미 할당된 주소 하나 선택, 최종 검토 및 확인의 네 단계로 진행합니다. 확인해도 이 모드에서는 저장하거나 설치하지 않으며, 취소하거나 입력이 종료되어도 아무것도 변경하지 않습니다.

```
sudo bash install.sh --pre-scan --configure-network
```

공개키, fingerprint, 접속 이름, 인증서와 개인키를 한 번에 넘기면 설치가 끝까지 진행됩니다. 실제 터미널에서 일반 설치를 실행하면 위 주소 선택 절차를 자동으로 제안하며, --configure-network 로 명시적으로 요청할 수도 있습니다. 기존 설치의 바인딩을 유지할 수 있습니다. --fingerprint 에는 패키지와 같은 채널이 아니라 별도 인증 채널로 받은 값을 넣습니다.

```
sudo bash install.sh \
  --release-key /root/blacklist-<버전>-release-public-key.asc \
  --fingerprint <별도 채널로 확인한 fingerprint> \
  --server-name blacklist.example.com \
  --tls-cert /root/tls/server.crt \
  --tls-key /root/tls/server.key
```

무인 설치는 `--interface IFACE`, `--bind-address IP` 로 대상을 지정합니다. 인터페이스에 사용 가능한 주소가 여러 개면 `--bind-address` 도 필요합니다. 아래 예시의 인터페이스와 문서용 IP 는 실제 할당된 값으로 바꾸고, 첫 설치에는 위 명령의 공개키·인증서 인자도 함께 넘깁니다. 기존 설치에서는 저장된 설정을 사용할 수 있습니다.

```
sudo bash install.sh --interface enp1s0 --bind-address 192.0.2.10 </dev/null
sudo bash install.sh --bind-address 192.0.2.10 </dev/null
```

기존 무인 실행 명령과 `/etc/blacklist/.env` 의 `FRONTEND_BIND_ADDRESS` 도 계속 지원합니다. 별도 선택이나 기존 설정이 없는 새 무인 설치는 종전처럼 `0.0.0.0` 을 사용합니다. 선택한 주소는 실제 설치에서만 `.env` 에 저장하고 프론트엔드의 호스트 443 포트 공개에만 적용합니다. 설치기는 호스트 NIC 의 주소, 링크, 라우팅, 방화벽 설정을 직접 재설정하거나 다른 프로젝트의 컨테이너를 중지하지 않습니다. 애플리케이션 브리지와 포트 공개에 필요한 네트워크 처리는 Docker 가 관리합니다.

설치기는 다음 순서로 진행합니다.

1. 네트워크 사전 점검과 공개키 fingerprint, 접속 이름, 인증서 유효기간, 개인키 일치, SAN 포함 여부 확인을 먼저 수행합니다. 부족한 항목이 있으면 모두 출력하고 아무것도 변경하지 않은 채 종료합니다.
2. 공개키를 `/etc/blacklist/release-pubkey.gpg` 에 등록합니다. 다른 키가 이미 등록돼 있으면 교체하지 않고 중단합니다.
3. 매니페스트 서명과 이미지 체크섬을 검증한 뒤 이미지를 로드합니다.
4. `/etc/blacklist/.env` 에 없는 관리자 비밀번호와 암호화 키를 생성하고, 인증서와 개인키를 `/etc/blacklist/frontend-tls` 에, 접속 이름을 `/etc/blacklist/.env` 에 기록합니다.
5. 서비스를 시작하고 다섯 컨테이너가 모두 healthy 가 될 때까지 기다립니다.

관리자 비밀번호는 화면에 출력하지 않습니다. 시크릿을 생성하는 즉시 `/etc/blacklist/.env.initial-admin-password` 에 권한 `0600` 으로 저장합니다. 이 파일에서 비밀번호 관리자로 가져온 뒤 파일을 삭제하고 관리자 화면에서 비밀번호를 변경하십시오. `/etc/blacklist/.env` 의 `ADMIN_PASSWORD` 는 최초 DB 초기화용이므로, 비밀번호를 변경한 뒤 이 값도 덮어쓰십시오.

관리자 계정이나 REGTECH 로그인을 직접 정하려면 첫 설치 전에 `/etc/blacklist/.env` 를 만들고 정할 값만 적습니다. 기존 배포가 없으면 설치기가 적힌 값을 그대로 두고 나머지 시크릿만 생성하며, 이 경우 초기 비밀번호 파일은 만들지 않습니다. `ADMIN_PASSWORD` 는 12 자 이상, 72 바이트 이하여야 하고 `$` 문자는 쓸 수 없습니다. `REGTECH_ID`/`REGTECH_PW` 는 둘 다 적거나 둘 다 비워 둡니다. 앱이 처음 기동할 때 저장된 REGTECH 로그인이 없으면 이 값을 암호화해 저장하고, 수집기가 최근 3 개월 최초 수집을 자동으로 시작합니다. 이후 변경은 관리자 화면에서 합니다.

```
sudo install -d -o root -g root -m 0700 /etc/blacklist
sudoedit /etc/blacklist/.env
```

```
ADMIN_USERNAME=admin
ADMIN_PASSWORD=<12 자 이상 비밀번호 >
REGTECH_ID=<REGTECH 아이디 >
REGTECH_PW=<REGTECH 비밀번호 >
WARP_ENABLED=false
```

같은 인자로 읽기 전용 사전 검증을 할 수 있습니다. 이 모드는 공개키를 호스트에 등록하지 않습니다.

```
sudo bash install.sh --verify-only --require-signature \
  --release-key /root/blacklist-<버전>-release-public-key.asc \
  --fingerprint <별도 채널로 확인한 fingerprint> \
  --server-name blacklist.example.com \
  --tls-cert /root/tls/server.crt \
  --tls-key /root/tls/server.key
```

4. 단계별 설치

원커맨드 대신 단계를 나눠 진행할 수도 있습니다. 먼저 공개키를 등록하고 fingerprint 를 별도 인증 채널의 값과 대조합니다. 출력된 fingerprint 가 일치하지 않으면 설치하지 않습니다.

```
gpg --batch --dearmor --output release-pubkey.gpg release-public-key.asc
gpg --show-keys --with-colons release-public-key.asc | awk -F: '$1 == "fpr" { print $10; exit }'
sudo install -d -o root -g root -m 0755 /etc/blacklist
```

```
sudo install -o root -g root -m 0644 release-pubkey.gpg /etc/blacklist/release-pubkey.gpg
rm -f release-pubkey.gpg
```

다음으로 시크릿을 생성하고, 인증서와 접속 이름을 준비한 뒤 설치기를 실행합니다.

```
sudo bash install.sh --check-secrets
sudo install -d -o 1001 -g 1001 -m 0700 /etc/blacklist/frontend-tls
sudo install -o 1001 -g 1001 -m 0600 server.key /etc/blacklist/frontend-tls/server.key
sudo install -o 1001 -g 1001 -m 0644 server.crt /etc/blacklist/frontend-tls/server.crt
sudoedit /etc/blacklist/.env
sudo bash install.sh
```

`.env` 에서 실제 접속 FQDN 또는 IP 를 설정합니다. 인증서 SAN 은 이 값과 일치해야 합니다.

```
FRONTEND_TLS_MODE=provided
FRONTEND_TLS_SERVER_NAME=blacklist.example.com
FRONTEND_BIND_ADDRESS=0.0.0.0
```

호스트와 다른 Docker 네트워크의 서브넷이 겹치면 `/etc/blacklist/.env` 에서 다음 세 값을 함께 조정하고 `--pre-scan` 을 다시 실행합니다. 이 값들은 외부 접속용 `FRONTEND_BIND_ADDRESS` 와 별개인 Docker 내부 통신 설정입니다.

설정	조건
`BLACKLIST_NETWORK_SUBNET`	호스트 및 다른 Docker 네트워크와 겹치지 않는 애플리케이션 브리지 서브넷 (CIDR)
`FRONTEND_INTERNAL_IP`	위 서브넷 안에서 프론트엔드가 사용할 유효한 주소. 브리지 게이트웨이 주소는 제외
`TRUSTED_PROXY_NETWORKS`	위 프론트엔드 내부 주소를 가리키는 `/32` 신뢰 프록시 범위

개발용 자체서명 모드는 명시적으로만 사용할 수 있고 installer 가 loopback bind 를 강제합니다.

```
FRONTEND_TLS_MODE=self-signed
FRONTEND_BIND_ADDRESS=127.0.0.1
```

5. 설치 결과 확인

```
sudo docker compose --env-file /etc/blacklist/.env -f docker-compose.yml ps
curl --insecure --fail https://blacklist.example.com/health
```

상태 확인 URL 은 선택한 바인딩 주소로 연결되는 실제 접속 FQDN 또는 IP 로 바꿉니다.

정상 상태는 다음과 같습니다.

- `blacklist-app`, `blacklist-collector`, `blacklist-frontend`, `blacklist-postgres`, `blacklist-redis` 가 모두 healthy
- 호스트 공개 포트는 frontend 의 443 하나
- PostgreSQL 5432, Redis 6379, Collector 8545, Flask 2542 는 호스트에 미공개
- 미인증 보호 API 는 HTTP 401 반환

6. WARP 프록시

WARP 는 Collector 의 REGTECH 요청에만 쓰는 선택 기능입니다. 실운영 설치에는 필요하지 않으며 기본값은 꺼짐입니다. 설치 서버의 공인 IP 가 REGTECH 에서 차단될 때만 `/etc/blacklist/.env` 에서 켭니다.

```
WARP_ENABLED=true
WARP_PROXY_URL=
```

`WARP_PROXY_URL` 이 비어 있으면 Docker 호스트 게이트웨이인 `http://host.docker.internal:40000` 을 사용합니다. 프록시는 Docker 브리지에서 접근할 수 있는 주소 (예: `172.17.0.1:40000`) 에서 수신해야 하며, `127.0.0.1` 에만 바인딩된 프록시는 릴레이가 필요합니다. 설치기는 재실행해도 이 두 값을 바꾸지 않고, `WARP_ENABLED` 가 `true`/`false` 가 아니거나 URL 형식이 잘못되면 중단합니다. 값을 바꾼 뒤에는 설치기를 다시 실행하거나 Collector 만 다시 만듭니다.

```
sudo docker compose --env-file /etc/blacklist/.env -f docker-compose.yml up -d blacklist-collector
```

`make dev` 는 `WARP_ENABLED=true` 가 기본값입니다.

7. 업그레이드

새 패키지를 별도 디렉터리에 풀고 새 `install.sh` 를 실행합니다. 기존 `/etc/blacklist/.env`, `/etc/blacklist/release-pubkey.gpg`, 프론트엔드 인증서, Docker 볼륨을 유지해야 저장된 자격증명을 계속 복호화하고 릴리스 서명을 검증할 수 있습니다. 기존 env 에 `FRONTEND_TLS_MODE` 가 없으면 installer 가 `provided` 로 마이그레이션하므로, 실행 전에 server

certificate 와 FRONTEND_TLS_SERVER_NAME 을 준비해야 합니다. 설치기는 인증서 SAN 이 FRONTEND_TLS_SERVER_NAME 을 포함하는지 확인하고, 포함하지 않으면 아무것도 변경하지 않고 중단합니다.

```
cd blacklist-<새 버전>
sudo bash install.sh
```

인증서를 교체할 때는 --server-name, --tls-cert, --tls-key 를 함께 넘깁니다. 기존 바인딩은 유지할 수 있으며, 네트워크 사전 점검은 업그레이드 대상 프로젝트의 기존 프론트엔드가 공개한 443 포트를 자체 사용으로 인식합니다. 다른 프로젝트의 컨테이너를 중지할 필요는 없습니다.

8. 데이터 포함 패키지

이미 수집한 데이터를 담아 배포하면 새 설치에서 최근 3 개월 수집을 다시 하지 않아도 됩니다. 데이터가 있는 설치 호스트의 패키지 디렉터리에서 blacklist_ips 를 내보냅니다.

```
sudo bash install.sh --export-seed-data /root/blacklist-seed.csv.gz
```

서비스 이미지가 있는 저장소 체크아웃에서 그 파일을 넣어 패키지를 다시 만들고, 서명한 뒤 ZIP 으로 묶습니다.

```
python3 scripts/build_offline_bundle.py --output /root/blacklist-data --seed-data /root/blacklist-seed.csv.gz
gpg --armor --detach-sign --local-user <fingerprint> \
  --output /root/blacklist-data/blacklist-<버전>/MANIFEST.sha256.asc \
  /root/blacklist-data/blacklist-<버전>/MANIFEST.sha256
cd /root/blacklist-data && zip -qr blacklist-<버전>-data.zip blacklist-<버전>
```

데이터 파일은 패키지의 seed/blacklist_ips.csv.gz 에 들어가며 MANIFEST 와 서명 검증 대상입니다. 패키지를 다시 만들면 MANIFEST.sha256 이 바뀌어 공식 릴리스 서명은 맞지 않으므로, 설치 호스트에는 이 패키지에 서명한 키의 공개키와 fingerprint 를 --release-key, --fingerprint 로 등록합니다. 다른 릴리스 키가 이미 등록된 호스트에서는 서명 검증에 실패해 설치되지 않습니다.

설치기는 DB 가 비어 있을 때만 앱과 수집기를 시작하기 전에 이 데이터를 가져오고, 데이터가 이미 있는 DB 는 건드리지 않습니다. 가져온 데이터가 있으므로 최초 3 개월 수집은 건너뛰고 일일 수집만 이어집니다.